



API 명세서_V2

☰ 상태 완료

학원 학생관리 시스템 API 설계 문서

항목	내용
문서명	학원 학생관리 웹 시스템 API 설계 문서
버전	v1.2
작성일	2026-08-19 (v1.1 개정: 2026-08-27, v1.2 개정: 2026-08-27)
대상 독자	외주 개발사 / 개발 담당자
관련 문서	요구사항 명세서 v1.2, ERD v1.1, 화면 아키텍처, 화면 세트(관리자/선생님/학부모·학생)
문서 상태	초안 (Draft) — 실제 구현 시 필드/경로가 조정될 수 있음

1. 개요

1.1 목적

본 문서는 「학원 학생관리 웹 시스템 요구사항 명세서 v1.2」와 ERD v1.1을 기준으로, 프론트엔드(선생님/관리자 웹, 학부모·학생 웹)가 백엔드와 통신할 REST API의 자원(Resource), 엔드포인트, 요청/응답 형식을 정의한다.

1.2 설계 원칙

API는 자원 중심(REST) 설계를 따른다. URL은 명사형 복수 자원명을 사용하고(`/students`, `/classes`), 행위는 HTTP 메서드(GET/POST/PATCH/DELETE)로 표현한다. 응답은 JSON을 기본으로 하며, 날짜·시간은 ISO 8601(`YYYY-MM-DD`, `YYYY-MM-DDTHH:mm:ssZ`)을 사용한다. 인증된 사용자의 역할(admin/teacher/parent/student)에 따라 서버 단에서 접근 권한을 강제하며(NFR-05), 프론트엔드의 화면 분기와 무관하게 API 레벨에서도 동일한 권한 검증을 수행한다.

일부 화면(출석 체크, 숙제 체크, 테스트 체크)은 반 단위로 여러 학생의 데이터를 한 번에 저장해야 하므로(FR-02-02, FR-03-05, FR-04-04), 해당 자원에는 `bulk` 저장용 엔드포인트를 별도로 둔다.

2. 공통 사항

2.1 Base URL & 버전

```
https://api.academy.app/v1
```

모든 엔드포인트는 위 Base URL을 기준으로 한다. 이후 표에서는 `/v1` 을 생략하고 표기한다.

2.2 인증

`Authorization: Bearer {accessToken}` 헤더로 인증한다. 토큰은 로그인 시 발급되며, payload에 `userId`, `role` 을 포함해 서버가 매 요청마다 역할 기반 접근 제어(RBAC)를 수행할 수 있게 한다. (REQ-AUTH-01)

2.3 공통 응답 포맷

성공(단건)

json

```
{  "data": {  "..." : "..."  }}
```

성공(목록, 페이지네이션 포함)

json

```
{  "data": [ {  "..." : "..."  } ],  "meta": {  "page": 1,  "perPage": 20,  "total": 128  }}
```

실패

json

```
{  "error": {    "code": "FORBIDDEN_SCOPE",    "message": "담당하지 않은 반의 데이터에는 접근할 수 없습니다."  }}
```

2.4 공통 쿼리 파라미터 (목록 조회 API)

파라미터	설명	예시
<code>page</code> , <code>perPage</code>	페이지네이션	<code>?page=1&perPage=20</code>
<code>sort</code>	정렬 필드(- 는 내림차순)	<code>?sort=-createdAt</code>
<code>keyword</code>	검색어(이름 등)	<code>?keyword=김민준</code>

파라미터	설명	예시
<code>classId</code> , <code>studentId</code> , <code>from</code> , <code>to</code>	도메인별 필터	각 API 표 참고

2.5 공통 HTTP 상태 코드

코드	의미
200	조회/수정 성공
201	생성 성공
204	삭제 성공(본문 없음)
400	요청 형식 오류(필수값 누락 등)
401	인증 실패(토큰 없음/만료)
403	권한 없음(역할 또는 담당 범위 밖)
404	자원 없음
409	중복/충돌(예: 이미 등록된 아이디)
422	유효성 검증 실패
500	서버 오류

2.6 공통 에러 코드(발체)

코드	상황
<code>UNAUTHENTICATED</code>	로그인이 필요함
<code>FORBIDDEN_ROLE</code>	해당 역할은 접근할 수 없는 API
<code>FORBIDDEN_SCOPE</code>	역할은 맞으나 담당 반/자녀 범위를 벗어난 요청 (예: 선생님이 비담당 반 출석 조회)
<code>VALIDATION_ERROR</code>	입력값 검증 실패, <code>details</code> 에 필드별 오류 포함
<code>NOT_FOUND</code>	자원이 존재하지 않음
<code>DUPLICATE_LOGIN_ID</code>	로그인 아이디 중복

3. 인증 API

Method	Path	설명	권한
POST	<code>/auth/login</code>	로그인	전체
POST	<code>/auth/logout</code>	로그아웃(토큰 폐기)	로그인 사용자
POST	<code>/auth/refresh</code>	Access Token 갱신	로그인 사용자(Refresh Token 보유)

Method	Path	설명	권한
POST	/auth/password/reset-request	비밀번호 재설정 이메일/코드 발송	전체
POST	/auth/password/reset	비밀번호 재설정 확정	전체
GET	/me	내 계정 정보 조회	로그인 사용자

POST /auth/login 요청

json

```
{ "loginId": "jihyun", "password": "*****" }
```

응답 — role 에 따라 프론트엔드가 SCR-02/03/12로 자동 분기한다(전체 흐름 다이어그램 참고).

json

```
{ "data": { "accessToken": "eyJhbGciOi...", "refreshToken": "eyJhbGciOi...", "user": { "id": 12, "name": "박지현", "role": "teacher", "hasMultipleChildren": false } } }
```

학부모 계정이면서 자녀가 2명 이상인 경우 hasMultipleChildren: true 가 내려가며, 프론트엔드는 홈(SCR-12) 대신 자녀 선택(SCR-17)으로 먼저 이동한다. (FR-AUTH-03)

4. 계정 관리 API — 관리자 전용 (SCR-06)

계정 관리는 선생님/학부모/학생 3개 역할을 모두 포함한다(v1.2, REQ-AUTH-05·FR-01-07). 학생 계정은 통상 SCR-04 학생 등록 시 함께 발급되지만(5장 참고), 아이디 재발급·초기화 등 계정 자체의 관리는 본 SCR-06/ /users API로 이루어진다.

Method	Path	설명
GET	/users?role=teacher parent student	계정 목록(역할별 탭)
POST	/users	선생님/학부모/학생 계정 생성
GET	/users/{id}	계정 상세
PATCH	/users/{id}	계정 정보 수정
PATCH	/users/{id}/status	활성/비활성 전환
POST	/users/{id}/reset-password	비밀번호 초기화(관리자용, FR-01-07) — 임시 비밀번호를 재발급하고 최초 로그인 시 변경을 강제하는 플래그를 함께 내려준다

POST `/users` 요청 예시(선생님 생성)

json

```
{  "role": "teacher",  "name": "이수민",  "loginId": "sumin",  "phone": "010-0000-0000",  "classIds": [3, 4]}
```

5. 학생 관리 API (SCR-04, SCR-11)

Method	Path	설명	권한
GET	<code>/students?</code> <code>classId=&status=&keyword=</code>	학생 목록/검색	admin(전체), teacher(담당 반만)
POST	<code>/students</code>	학생 등록(학부모 계정 동시 생성 + 학생 본인 로그인 계정 발급 포함, v1.2)	admin
GET	<code>/students/{id}</code>	학생 상세	admin, teacher(담 당), parent/student(본인· 자녀)
PATCH	<code>/students/{id}</code>	학생 정보 수정(반 배정, 상태 변경 등)	admin
GET	<code>/students/{id}/summary</code>	출석·숙제·테스트·모의 고사 통합 요약(SCR- 11)	admin, teacher(담당)

POST `/students` 요청 예시 — `account.loginId` 를 생략하면 서버가 아이디를 자동 생성한다 (FR-01-07). 응답에는 발급된 임시 비밀번호가 1회에 한해 포함된다.

json

```
{  "name": "김민준",  "birthDate": "2012-05-14",  "school": "○○중",  "grade": "중2",  "classId": 3,  "account": {    "loginId": "kimminjun01",    "autoGenerateLoginId": false  },  "parent": {    "createNew": true,    "name": "김민준 학부모",    "phone": "010-1111-2222",    "loginId": "kimparent01",    "relationType": "mother"  }}
```

GET `/students/{id}/summary` 응답 예시

json

```
{  "data": {    "student": { "id": 101, "name": "김민준", "grade": "중2", "className": "중2 심화반" },    "attendance": { "presentDays": 18, "totalDays": 20, "lateCount": 1, "absentCount": 0 },    "homework": [ { "title": "구문 노트 정리", "isDone": false, "date": "2026-08-17" } ],    "recentTest": { "sessionDate": "2026-08-19", "scores": { "vocab": 18, "reading": 16, "grammar": 11, "syntax": 15 } },    "recentMonthlyExam": { "examMonth": "2026-08", "rawScore": 82, "deltaFromPrev": -2 }  } }
```

6. 반 관리 API (SCR-05)

Method	Path	설명	권한
GET	/classes	반 목록	admin(전체), teacher(담당만)
POST	/classes	반 생성	admin
GET	/classes/{id}	반 상세(시간표, 담당 선생님)	admin, teacher(담당)
PATCH	/classes/{id}	반 정보 수정	admin
DELETE	/classes/{id}	반 삭제(소속 학생 없을 때만)	admin
GET	/classes/{id}/students	반 소속 학생 목록(Class 1:N Student, v1.1에서 관계 명시)	admin, teacher(담당)

7. 선생님 배정 API

Method	Path	설명
GET	/teacher-assignments? teacherId=	배정 목록
POST	/teacher-assignments	반 또는 학생 단위 배정 추가 (classId 또는 studentId 중 하나)
DELETE	/teacher-assignments/{id}	배정 해제

8. 학부모·자녀 연결 API (SCR-17)

Method	Path	설명	권한
GET	/me/children	로그인한 학부모의 자녀 목록	parent

Method	Path	설명	권한
POST	/parent-links	학부모-학생 연결 생성(부/모 구분 포함)	admin
PATCH	/parent-links/{id}	연결 정보 수정(부/모 구분 변경)	admin
DELETE	/parent-links/{id}	연결 해제	admin

POST /parent-links 요청 예시 — `relationType` 으로 부/모/기타를 구분한다(v1.1 신규, ParentStudent 엔티티).

json

```
{ "parentUserId": 45, "studentId": 101, "relationType": "mother" }
```

GET /me/children 응답 예시 — 자녀 선택 화면(SCR-17)에서 바로 사용.

json

```
{ "data": [ { "studentId": 101, "name": "김민준", "grade": "중2", "className": "중2 심화반", "relationType": "mother" }, { "studentId": 102, "name": "김서윤", "grade": "초5", "className": "초등 문법반", "relationType": "mother" } ] }
```

자녀 선택 후 이후 모든 조회 API 호출 시 `X-Active-Student-Id` 헤더(또는 쿼리 파라미터 `studentId`)로 조회 대상을 지정하며, 서버는 해당 학생이 요청자의 자녀인지 매 요청마다 검증한다. (NFR-05)

9. 출석 API (SCR-07, SCR-13)

Method	Path	설명	권한
GET	/attendance?classId=&date=	특정 반·날짜의 출석 목록(체크 화면 초기 로딩용)	admin, teacher(담당)
POST	/attendance/bulk	반 단위 일괄 저장 (FR-02-02)	admin, teacher(담당)
PATCH	/attendance/{id}	개별 기록 수정	admin, teacher(담당)
GET	/students/{id}/attendance?month=2026-08	학생 월별 출석 조회 (캘린더 뷰)	admin, teacher(담당), parent/student(본인)

POST `/attendance/bulk` 요청

json

```
{  "classId": 3,  "date": "2026-08-19",  "records": [    { "studentId": 101, "status": "present", "note": null },    { "studentId": 102, "status": "late", "note": "15분 지각" },    { "studentId": 103, "status": "absent", "note": "병원 진료" }  ]}
```

`status` 허용값: `present` (출석) · `late` (지각) · `absent` (결석) · `earlyLeave` (조퇴) · `makeup` (보강). 상태값 목록은 관리자가 `/attendance-statuses` 로 관리할 수 있도록 확장 가능(FR-02-01 비고 참고).

10. 숙제 API (SCR-08, SCR-14)

Method	Path	설명	권한
GET	<code>/homework-items?classId=&week=</code>	반의 숙제 항목 목록	admin, teacher(담당)
POST	<code>/homework-items</code>	숙제 항목 생성(제목/범위/기간)	admin, teacher(담당)
PATCH	<code>/homework-items/{id}</code>	숙제 항목 수정	admin, teacher(담당)
DELETE	<code>/homework-items/{id}</code>	숙제 항목 삭제	admin, teacher(담당)
GET	<code>/homework-items/{id}/records</code>	항목별 학생 기록(체크 화면 로딩)	admin, teacher(담당)
POST	<code>/homework-records/bulk</code>	학생×항목 매트릭스 일괄 저장(FR-03-05)	admin, teacher(담당)
GET	<code>/students/{id}/homework?from=&to=</code>	학생 숙제 이력 조회	admin, teacher(담당), parent/student(본인)

POST `/homework-items` 요청

json

```
{  "classId": 3,  "title": "단어장 Ch.5",  "scope": "Unit 12-15",  "assignedDate": "2026-08-17",  "dueDate": "2026-08-19" }
```

POST `/homework-records/bulk` 요청 — 여러 숙제 항목을 한 번에 저장할 수 있도록 항목 배열을 받는다.

json

```
{ "classId": 3, "items": [ { "homeworkItemId": 501,
"records": [ { "studentId": 101, "isDone": true, "score": 98, "comment": "오타 1개" }, { "studentId": 102, "isDone": true, "score": 100, "comment": null } ] }, { "homeworkItemId": 502, "records": [ { "studentId": 101, "isDone": false, "score": null, "comment": "재제출 요청" } ] } ] }
```

11. 테스트 API (SCR-09, SCR-15)

Method	Path	설명	권한
GET	/test-sessions?classId=	테스트 회차 목록	admin, teacher(담당)
POST	/test-sessions	회차 생성	admin, teacher(담당)
GET	/test-sessions/{id}/records	회차의 학생별 4영역 성적(체크 화면 로딩)	admin, teacher(담당)
POST	/test-records/bulk	학생×영역 매트릭스 일괄 저장(FR-04-04)	admin, teacher(담당)
GET	/students/{id}/tests?limit=10	학생 테스트 이력 조회	admin, teacher(담당), parent/student(본인)

POST /test-records/bulk 요청 — subject 는 vocab (단어) · reading (독해) · grammar (문법) · syntax (구문독해) 중 하나이며, 영역별로 응시하지 않았으면 배열에서 생략한다(FR-04-02).

json

```
{ "testSessionId": 901, "records": [ { "studentId": 101,
"subject": "vocab", "isTaken": true, "score": 18, "maxScore": 20 }, { "studentId": 101, "subject": "reading", "isTaken": true, "score": 16, "maxScore": 20 }, { "studentId": 101, "subject": "grammar", "isTaken": true, "score": 11, "maxScore": 20 }, { "studentId": 101, "subject": "syntax", "isTaken": true, "score": 15, "maxScore": 20 } ] }
```

12. 월말모의고사 API (SCR-10, SCR-16)

Method	Path	설명	권한
GET	/monthly-exams	회차 목록	admin, teacher

Method	Path	설명	권한
POST	<code>/monthly-exams</code>	회차 생성(FR-05-01)	admin, teacher(담당)
GET	<code>/monthly-exams/{id}/records?classId=</code>	회차의 반별 성적 목록	admin, teacher(담당)
POST	<code>/monthly-exam-records</code>	학생 성적 등록(FR-05-02)	admin, teacher(담당)
PATCH	<code>/monthly-exam-records/{id}</code>	성적 수정	admin, teacher(담당)
GET	<code>/students/{id}/monthly-exams?limit=5</code>	학생 추이 조회(그래프용, FR-05-05)	admin, teacher(담당), parent/student(본인)
GET	<code>/monthly-exam-records/{id}</code>	회차 상세(성적 + 피드백 전체)	admin, teacher(담당), parent/student(본인)
POST	<code>/monthly-exam-records/{id}/type-feedbacks</code>	유형별 피드백 추가 (FR-05-08)	admin, teacher(담당)
PATCH	<code>/type-feedbacks/{id}</code>	유형별 피드백 수정	admin, teacher(담당)
DELETE	<code>/type-feedbacks/{id}</code>	유형별 피드백 삭제	admin, teacher(담당)
PUT	<code>/monthly-exam-records/{id}/score-feedback</code>	점수대별 피드백 등록/수정(upsert)	admin, teacher(담당)
GET	<code>/type-categories</code>	유형 카테고리 마스터 목록	전체(조회)
POST	<code>/type-categories</code>	유형 카테고리 추가 (FR-05-08)	admin

POST `/monthly-exam-records` 요청

json

```
{ "monthlyExamId": 12, "studentId": 101, "rawScore": 82, "stdScore": 128, "percentile": 91, "grade": "2등급" }
```

GET `/students/{id}/monthly-exams?limit=5` 응답 — SCR-16의 시계열 그래프에 그대로 사용.

json

```
{ "data": [ { "examMonth": "2026-04", "rawScore": 72 }, { "examMonth": "2026-05", "rawScore": 78 }, { "examMonth": "2026-06", "rawScore": 75 }, { "examMonth": "2026-07", "rawScore": 72 } ] }
```

```
Score": 84 }, { "examMonth": "2026-08", "rawScore": 82 }
]}
```

GET `/monthly-exam-records/{id}` 응답 — 회차 상세 화면(SCR-16) 전체를 한 번에 채운다.

json

```
{ "data": { "record": { "examMonth": "2026-08", "rawScore": 82, "percentile": 91, "grade": "2등급" }, "typeFeedbacks": [ { "id": 1, "typeCategory": "어휘", "status": "strength", "feedbackText": "어휘 문제 정답률 95%로 우수합니다." }, { "id": 2, "typeCategory": "빈칸추론", "status": "needsWork", "feedbackText": "반복적으로 오답이 발생해 추가 연습이 필요합니다." } ], "scoreFeedback": { "scoreBand": "80점대", "feedbackText": "기본 문법과 어휘는 탄탄하나..." } } }
```

13. 대시보드 API (SCR-02, SCR-03)

Method	Path	설명	권한
GET	<code>/dashboard/admin</code>	전체 재원생 수, 오늘 출석률, 숙제 미완료 건수, 반별 현황 등	admin
GET	<code>/dashboard/teacher</code>	담당 반의 오늘 할 일(출석/숙제/테스트 입력 여부), 반별 요약 + 학원 전체 반 통계(요약) (v1.1, FR-06-03)	teacher

`/dashboard/teacher` 응답에는 v1.1부터 `allClassesSummary` 필드가 추가되어, 선생님이 담당하지 않는 반을 포함한 학원 전체 반의 출석률·숙제 완료율 등 통계 요약을 함께 내려받는다. 다만 출석/숙제/테스트/월말모의고사 등 **실제 개별 데이터의 조회·입력·수정은 기존과 동일하게 담당 반으로 한정**되며, 이 필드는 집계된 통계값만 제공한다.

json

```
{ "data": { "myClasses": [ { "classId": 3, "className": "중2 심화반", "todayAttendanceRate": 0.95, "homeworkDoneRate": 0.88 } ], "allClassesSummary": [ { "classId": 1, "className": "초등 문법반", "todayAttendanceRate": 0.9, "homeworkDoneRate": 0.8 }, { "classId": 3, "className": "중2 심화반", "todayAttendanceRate": 0.95, "homeworkDoneRate": 0.88 } ] } }
```

14. 공지사항 API (SCR-18, SCR-19, FR-09)

Method	Path	설명	권한
GET	<code>/notices?</code> <code>scope=all class&classId=</code>	공지 목록(전체/반별 필터)	admin(전체), teacher(전체+담당 반)
POST	<code>/notices</code>	공지 작성	admin(전체 공지 가능), teacher(담당 반 공지 만)
GET	<code>/notices/{id}</code>	공지 상세	admin, teacher, parent/student(본인 자녀·본인 관련)
PATCH	<code>/notices/{id}</code>	공지 수정	admin(전체), teacher(본인이 작성한 담당 반 공지만)
DELETE	<code>/notices/{id}</code>	공지 삭제	admin(전체), teacher(본인이 작성한 담당 반 공지만)
PATCH	<code>/notices/{id}/pin</code>	상단 고정 토글 (<code>isPinned</code>)	admin, teacher(담당 반 공지만)
GET	<code>/me/notices</code>	로그인 사용자에게 노출 되는 공지 목록(전체 공 지 + 본인/자녀 반 공지)	parent, student

POST `/notices` 요청 예시 — `scope` 가 `class` 이면 `classId` 가 필수이며, 서버는 작성자가 해당 반 담당 선생님인지(또는 admin인지) 검증한다.

json

```
{  "scope": "class",  "classId": 3,  "title": "8/29(토) 특강 안  
내",  "content": "이번 주 토요일 특강은 2시간 진행됩니다...",  "isP  
inned": false}
```

GET `/me/notices` 응답 예시 — 학부모는 자녀가 여러 명이면 활성 자녀(`X-Active-Student-Id`)
기준 반 공지가 함께 내려간다.

json

```
{  "data": [    {  "id": 21,  "scope": "all",  "title": "추석 연휴  
휴원 안내",  "isPinned": true,  "createdAt": "2026-08-25T09:00:00  
Z"  },    {  "id": 24,  "scope": "class",  "className": "중2 심화
```

```
반", "title": "8/29(토) 특강 안내", "isPinned": false, "createdAt": "2026-08-27T10:30:00Z" } ]}
```

15. 권한 매트릭스 (엔드포인트 그룹 기준)

자원 그룹	admin	teacher	parent	student
계정 관리 (<code>/users</code>)	CRUD	-	-	-
학생/반 관리 (<code>/students</code> , <code>/classes</code>)	CRUD	R(담당만)	-	-
출석/숙제/테스트 입력 (<code>*/bulk</code>)	CRUD(전체)	CRUD(담당만)	-	-
출석/숙제/테스트 조회 (<code>/students/{id}/*</code>)	R(전체)	R(담당만)	R(자녀만)	R(본인만)
월말모의고사 입력·피드백	CRUD(전체)	CRUD(담당만)	-	-
월말모의고사 조회	R(전체)	R(담당만)	R(자녀만)	R(본인만)
공지사항 (<code>/notices</code>)	CRUD(전체)	CRUD(담당 반 공지만)※	R(본인 자녀 관련)	R(본인 관련)
대시보드	전체 통계	담당 반 통계 + 전체 반 통계 조회 ※※(v1.1)	-	-

※ 선생님은 담당 반 대상 공지만 작성·수정·삭제할 수 있으며, 학원 전체 대상(`scope=all`) 공지하는 admin만 작성할 수 있다.

※※ 선생님의 전체 반 통계 접근은 대시보드 집계 조회(`GET /dashboard/teacher`)에 한정되며, 비담당 반의 개별 학생 데이터(`/students/{id}/*`)에는 여전히 접근할 수 없다.

모든 "담당만" / "자녀만" / "본인만" 범위 제한은 응답 데이터 필터링이 아니라 **요청 단계에서 403 FORBIDDEN_SCOPE** 로 거부하는 것을 원칙으로 한다. (NFR-05)

16. 부록

16.1 주요 Enum 정의

Enum	값
<code>role</code>	<code>admin</code> · <code>teacher</code> · <code>parent</code> · <code>student</code>
<code>student.status</code>	<code>enrolled</code> (재원) · <code>paused</code> (휴원) · <code>withdrawn</code> (퇴원)

Enum	값
<code>attendance.status</code>	<code>present</code> · <code>late</code> · <code>absent</code> · <code>earlyLeave</code> · <code>makeup</code>
<code>homeworkRecord.isDone</code>	<code>true</code> · <code>false</code>
<code>testRecord.subject</code>	<code>vocab</code> · <code>reading</code> · <code>grammar</code> · <code>syntax</code>
<code>typeFeedback.status</code>	<code>strength</code> (강점) · <code>needsWork</code> (보완 필요)
<code>parentStudent.relationType</code>	<code>father</code> (부) · <code>mother</code> (모) · <code>other</code> (기타) — v1.1 신규
<code>notice.scope</code>	<code>all</code> (전체 공지) · <code>class</code> (반별 공지) — v1.1 신규

16.2 확인 필요 사항

- ☐ 인증 방식: JWT 자체 발급 vs. 외부 Auth 서비스(Auth0, Firebase Auth 등) 사용 여부
- ☐ 토큰 만료 시간 및 Refresh Token 정책
- ☐ 반별 다건 동시 저장(`bulk`) 시 일부 실패 처리 방식(전체 롤백 vs. 부분 성공)
- ☐ 파일/이미지 업로드 필요 여부(현재 문서 범위에는 없음)
- ☐ 실시간 반영 방식: 단순 재조회(polling)로 충분한지, 웹소켓 등 실시간 갱신이 필요한지 (FR-07-04는 "재접속/새로고침 시 반영"을 최소 요건으로 명시)

16.3 변경 이력

버전	일자	내용
v1.0	2026-08-19	최초 작성
v1.1	2026-08-27	① <code>/parent-links</code> 에 <code>relationType</code> (부/모/기타) 추가, <code>/me/children</code> 응답에 반영 ② <code>/classes/{id}/students</code> 에 Class-Student 1:N 관계 명시 ③ 공지사항 API(<code>/notices</code> , <code>/me/notices</code>) 신설(14장) ④ <code>/dashboard/teacher</code> 에 <code>allClassesSummary</code> (전체 반 통계 조회) 추가 ⑤ 권한 매트릭스·Enum정의 갱신, 요구사항 명세서 v1.1 반영
v1.2	2026-08-27	⑥ 계정 관리 API(4장)에 학생 역할 추가(<code>role=student</code> 조회, 학생 계정 생성) ⑦ <code>POST /users/{id}/reset-password</code> 신설 (FR-01-07) ⑧ <code>POST /students</code> 요청에 학생 본인 로그인 계정 발급 필드(<code>account</code>) 추가, 요구사항 명세서 v1.2 반영

본 문서는 초안이며, 실제 구현 시 개발사의 기술 스택(프레임워크, ORM 등)에 맞춰 세부 경로·필드명이 조정될 수 있습니다.